

# MUPC 2026 Solution

by

MUPC 2026 개최진



## Staff

### 개최

✓ kckyoung2      명지대학교

### 출제

✓ dustin36      선도소프트

### 운영

✓ BUILD      단과대 학생회



## Supporter

### 선제

✓ bnb2011      삼성전자

### 검수

✓ kky085437      고려대학교

✓ yyyy7089      경희대학교

✓ kcits970      단국대학교

✓ gmb9817      대전대신고

✓ chleee      숭실대학교

✓ jms020820      숭실대학교

✓ m4ushold      숭실대학교

✓ nflight11      연세대학교

✓ sadtreap

✓ eka

### 테스터

✓ woohyun\_jng      gshs

✓ minjunnick      NLCS Jeju

✓ sub0411      부산대학교

✓ 7bellaa      부산대학교

✓ daehee      부산대학교



✓ dinojaemin



## Special Thanks

- ✓ IBory                      대회 후원
- ✓ VOID                      대회 후원
- ✓ wapas                    typst 양식 제공



Sponsor



명지대학교 인공지능·소프트웨어융합대학

***Jungbi***



| 번호 | 문제                   | 난이도    | 출제자       |
|----|----------------------|--------|-----------|
| A  | ATRI                 | Medium | kckyoung2 |
| B  | Bit parity game      | Easy   | kckyoung2 |
| C  | Contest Title        | Easy   | kckyoung2 |
| D  | Dr. Oh               | Easy   | dustin36  |
| E  | Electric Pylon       | Easy   | kckyoung2 |
| F  | Flathead grey mullet | Medium | dustin36  |
| G  | GO! GO! MARU         | Medium | kckyoung2 |
| H  | Handle's rule        | Hard   | kckyoung2 |
| I  | I'm fox, not cat     | Hard   | kckyoung2 |
| J  | Jaguar               | Easy   | dustin36  |
| K  | kENN3N's Trekking    | Hard   | kckyoung2 |

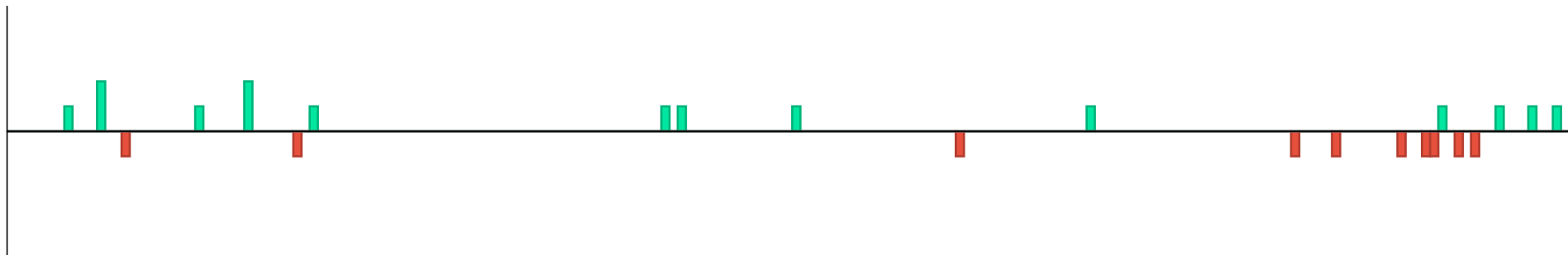


## A. ATRI

#dp

출제진 의도 - **Medium**

- ✓ 출제자: kckyoung2
- ✓ 제출: 26번, 정답: 15명 (정답률 57.7%), 처음 푼 사람: plast, 7분







## A. ATRI

- ✓ Subtask 1 (5점):  $N \leq 2$ 
  - ✓ 아래와 같은 게임판에서 1번 칸과 4번 칸은 앞면과 뒷면 중 큰 쪽을 선택하는 것이 게임판의 점수를 더 크게 만들 수 있습니다.
  - ✓ 따라서 2번 칸과 3번 칸을 포함하는 대각선을 뒤집는 경우와 그렇지 않은 경우 2가지에 대해서 게임판의 점수를 계산하고, 그 중 더 최대가 되는 값이 정답이 됩니다.

|   |   |
|---|---|
| 1 | 2 |
| 3 | 4 |

- ✓ Subtask 2 (50점):  $N \leq 10$ 
  - ✓ Subtask 1의 풀이를 일반화 해서 모든 대각선을 뒤집는  $2^{2N-1}$  가지의 경우에서 게임판의 점수가 최대가 되는 값이 정답이 됩니다.
  - ✓ 따라서 비트마스크 + 완전탐색으로  $O(2^N N^2)$ 에 해결할 수 있습니다



## A. ATRI

- ✓ Subtask 3 (30점):  $N \leq 300$ 
  - ✓ 어떤 칸  $(i, j)$ 에서 오른쪽 위를 향하는 대각선의 인덱스를  $i + j$ 와 같이 나타낼 수 있습니다.
  - ✓ 현재 대각선  $k$ 를 뒤집었을 때 영향을 받는 칸은  $k - 1$  대각선과  $k + 1$  대각선 위에 위치한 칸들이므로, 대각선 한 줄을 뒤집는 행동의 순서를 임의로 정할 수 있습니다.
  - ✓ 따라서, 이전 대각선과 현재 대각선의 앞뒷면을 나타내는 4가지의 상태가 존재합니다.
  - ✓  $dp[i][state]$ 와 같이 상태공간을 정의하고, 이전 대각선에서 다음 대각선으로 전이하면 됩니다.
  - ✓ 이전 대각선 판별을 비효율적으로 할 경우  $O(N^3)$ 에 해결할 수 있습니다.
- ✓ Subtask 4 (15점): 제한 조건 없음 (main solution)
  - ✓ Subtask 3에서 이전 대각선의 칸들을  $\{(i, j) \mid i + j = k - 1\}$ 로 볼 수 있으므로  $O(N^2)$ 에 해결할 수 있습니다.

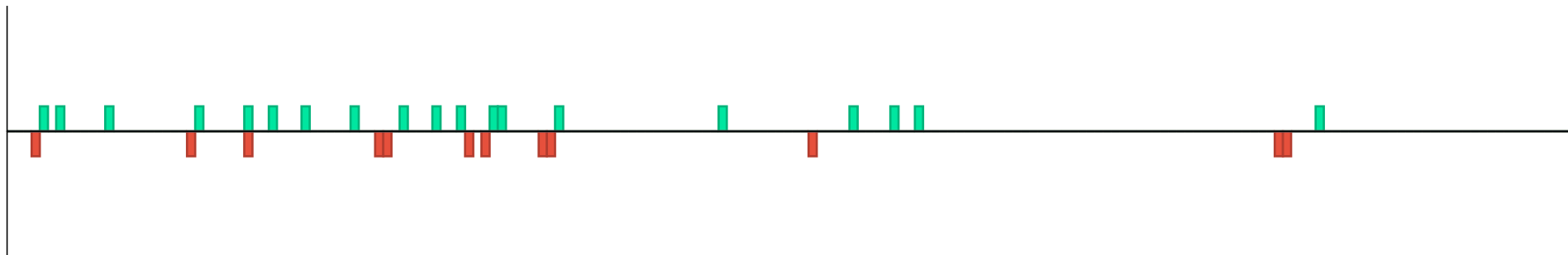


## B. Bit parity game

#game\_theory

출제진 의도 - **Easy**

- ✓ 출제자: kckyoung2
- ✓ 제출: 31번, 정답: 19명 (정답률 61.3%), 처음 푼 사람: akrobatik, 4분





## B. Bit parity game

- ✓ Subtask 1 (10점):  $N \leq 2$ 
  - ✓ 모든 경우의 수인 6가지에 대해서 전부 해보면 됩니다.
- ✓ Subtask 2 (10점): 모든 카드 값이 홀수
  - ✓ Subtask 1을 통해서 마지막 남은 카드에 쓰여져 있는 값이 홀수일 경우 홀수와 짝수 둘 다 만들 수 있다는 사실을 알 수 있습니다.
  - ✓ 따라서 마지막 턴을 진행하는 팀이 무조건 승리합니다.
- ✓ Subtask 3 (10점): 모든 카드 값이 짝수
  - ✓ 모든 카드 값이 짝수일 경우 어떤 비트 연산자를 사용해도 항상 짝수가 됩니다.
  - ✓ 따라서 hambak이 무조건 승리합니다.



## B. Bit parity game

- ✓ Subtask 4 (70점): 제한 조건 없음 (main solution)
  - ✓ Subtask 1, 2, 3을 통해 hambak이 승리하기 위한 조건과 myongjin이 승리하기 위한 조건을 알 수 있습니다.
  - ✓ myongjin이 승리하기 위해서는  $n$ 이 홀수이면서, 홀수를 최대한 많이 남겨, 마지막 턴에 홀수를 사용할 수 있어야 합니다.
  - ✓ 즉, 홀수의 개수가 짝수의 개수보다 많으면서,  $n$ 이 홀수이면 myongjin이 승리하고, 그렇지 않은 모든 경우에 hambak이 승리합니다.



## C. Contest Title

#implementation

출제진 의도 - **Easy**

- ✓ 출제자: kckyoung2
- ✓ 제출: 36번, 정답: 26명 (정답률 72.2%), 처음 푼 사람: yoni1224, 4분





## C. Contest Title

- ✓ map 등의 자료구조를 사용해서 개수를 세주면 됩니다.
- ✓ Python에서는 Dictionary를 사용하면 됩니다.
- ✓ MUPC, MJUPC, MPC, MJPC를 제외한 문자열이 들어올 경우 MUPC의 개수를 1 증가시키면 됩니다.

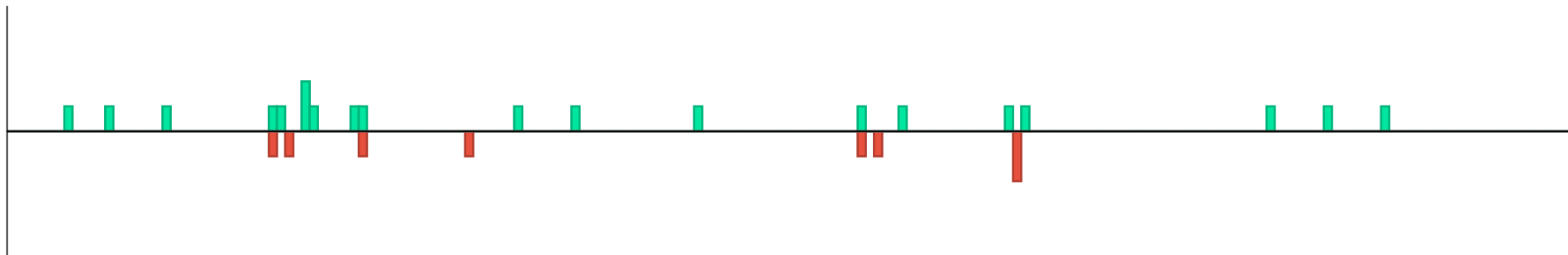


## D. Dr. Oh

#math, #greedy, #sorting

출제진 의도 - **Easy**

- ✓ 출제자: dustin36
- ✓ 제출: 28번, 정답: 20명 (정답률 71.4%), 처음 푼 사람: yoni1224, 7분







## D. Dr. Oh

- ✓ 모든 후보의 중요도는 양수입니다. 따라서  $K$ 개를 고른다고 했을 때 선택한 중요도의 합을 최대로 만들려면, 중요도가 큰 후보부터 고르는 것이 항상 최적입니다.
- ✓ 즉,  $w_i = 2^{a_i}$ 들을 내림차순으로 정렬한 뒤, prefix sum이 전체 합의  $\frac{P}{Q}$  이상이 되는 최초의 위치를 찾으면 됩니다.
- ✓ 다만  $a_i$ 는 매우 클 수 있으므로  $2^{a_i}$ 를 그대로 계산하면 안 됩니다. 모든  $a_i$ 에서 최솟값  $m$ 을 빼도 모든 중요도가 같은 비율로 나누어지므로, 선택 비율은 변하지 않습니다.
- ✓ 조건에서 모든  $a_i$ 의 차이는 최대 20이므로,  $2^{(a_i-m)}$ 만 계산하면 충분합니다.
- ✓ 정렬된 배열을 앞에서부터 보며 현재까지 고른 합을  $S$ , 전체 합을  $T$ 라고 합시다. 최초로  $S \times Q \geq T \times P$ 가 되는 순간의 개수가 정답입니다.
- ✓ 중요도를 정렬한 후 선형적으로 탐색하여  $O(N \log N)$ 에 문제를 해결할 수 있습니다.

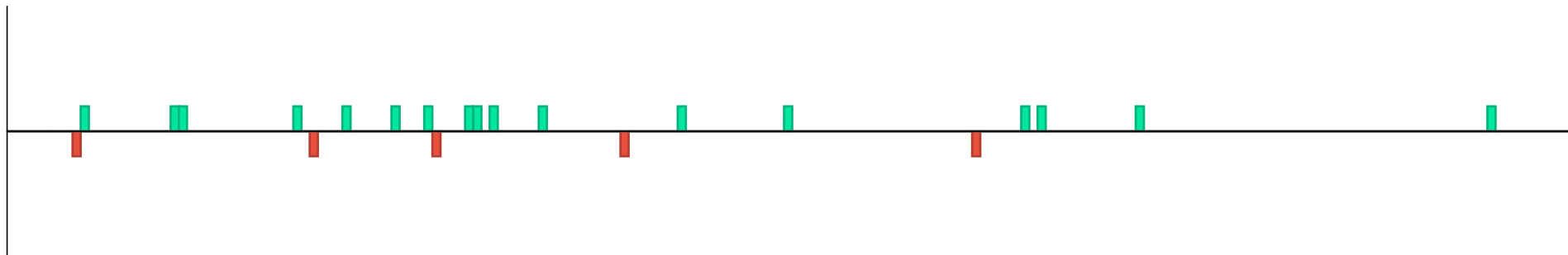


## E. Electric Pylon

#segtree, #data\_structures

출제진 의도 - Easy

- ✓ 출제자: kckyoung2
- ✓ 제출: 25번, 정답: 17명 (정답률 68.0%), 처음 푼 사람: basic, 9분





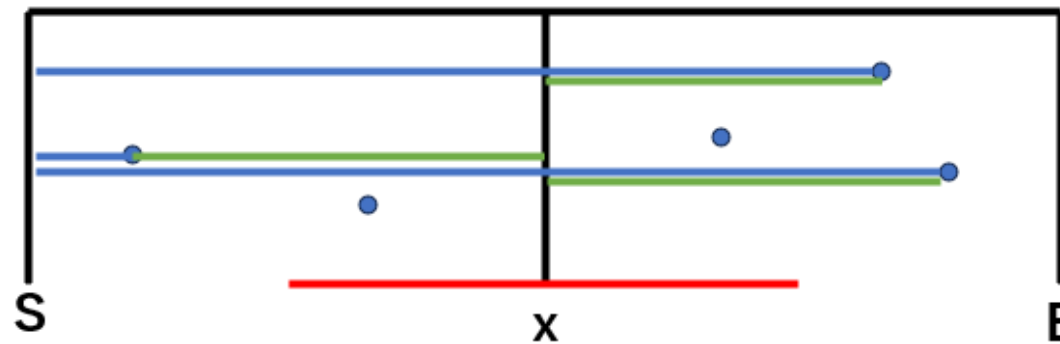
## E. Electric Pylon

- ✓ Subtask 1 (10점):  $N, Q \leq 1000$ 
  - ✓ cpp의 set 등을 이용해 1, 2번 작업을 처리합니다.
  - ✓ 3번 작업의 경우, 현재 설치된 모든 전동 채굴기와의 거리를 구하고  $K$ 보다 크다면 전부 더하면 됩니다.
  - ✓ 따라서  $O(Q \log N)$ 에 해결할 수 있습니다.
- ✓ Subtask 2 (40점): 3번 작업만 주어짐
  - ✓ 전동 채굴기가 새로 설치되거나 제거되지 않으므로 누적 합을 이용할 수 있습니다.
  - ✓ `precnt`, `presum`, `sufcnt`, `sufsum`을 각각 앞에서부터의 개수 합, 앞에서부터의 전동 채굴기와 0사이의 거리 합, 뒤에서부터의 개수 합, 뒤에서부터의 전동 채굴기와 200 000 사이의 거리 합이라고 합니다.
  - ✓ 3번 작업으로 들어온  $x$ 에 대해서 다음과 같이 계산할 수 있습니다.



## E. Electric Pylon

- ✓  $(\text{precnt}_{x-k-1} \times x - \text{presum}_{x-k-1}) + (\text{sufsum}_{x+k+1} - \text{sufcnt} \times x)$
- ✓ sum을 파란색으로,  $K$  값에 따라 전선을 연결하지 않아도 되는 범위를 빨간색으로, 구하는 거리를 초록색으로 나타냈을 때 그림은 아래와 같습니다.



- ✓ 따라서,  $O(N + Q)$ 에 해결할 수 있습니다.



## E. Electric Pylon

- ✓ Subtask 3 (50점): 제한 조건 없음 (main solution)
  - ✓ Subtask 2와 똑같은 방식으로 계산하되, 세그먼트 트리를 사용하여 설치 및 삭제를 처리하여 동적으로 3번 작업을 계산할 수 있습니다.
  - ✓ 따라서  $O(Q \log N)$ 에 해결할 수 있습니다.

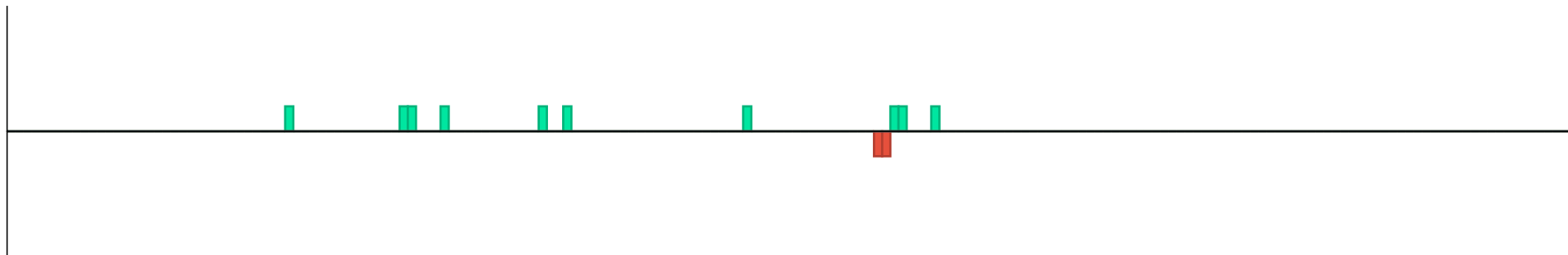


## F. Flathead grey mullet

#simulation, #data\_structures, #deque

출제진 의도 - **Medium**

- ✓ 출제자: dustin36
- ✓ 제출: 12번, 정답: 10명 (정답률 83.3%), 처음 푼 사람: akrobatik, 34분





## F. Flathead grey mullet

- ✓ Subtask 1 (20점):  $N \leq 2000$ 
  - ✓ 판매 과정에서 남은 송어들의 순서가 뒤집히는 시점은, 판매된 크기  $x$ 가 지금까지 판매된 값 중 최댓값이 되는 순간입니다.
  - ✓ 따라서 기록  $a_1, a_2, \dots, a_N$ 을 앞에서부터 보며 prefix maximum이 되는 값들을 미리 구해두면, 어느 턴에서 뒤집기가 일어났는지 알 수 있습니다.
  - ✓ 처음 진열 순서를 복원하기 위해 판매 과정을 거꾸로 시뮬레이션합니다. 마지막에 팔린 송어부터 덱에 다시 넣어가면 됩니다.
  - ✓ 한 턴을 거꾸로 되돌릴 때는 다음 순서로 처리합니다.
    - ✓ 판매 후 일어난 "맨 왼쪽 송어를 맨 오른쪽으로 옮기는 작업"을 되돌립니다. 이는 반대로 맨 오른쪽 송어를 맨 왼쪽으로 옮기는 작업입니다.
    - ✓ 해당 턴에 뒤집기가 일어났다면, 뒤집기를 다시 적용합니다.
    - ✓ 그 턴에 팔렸던 송어를 맨 왼쪽에 다시 넣습니다.



## F. Flathead grey mullet

- ✓ 이렇게 뒤집기가 일어날 때마다 적용하면 매 뒤집기에서  $O(N)$ 의 시간이 소요됩니다.
- ✓ 따라서  $O(N^2)$ 에 문제를 해결할 수 있습니다.
- ✓ Subtask 2 (80점): 제한 조건 없음 (main solution)
  - ✓ 매번 뒤집을 필요 없이 현재 방향이 뒤집혀 있는지를 나타내는 rev 플래그를 사용할 수 있습니다.
  - ✓ rev 값에 따라 앞과 뒤의 의미만 바꾸어 덱 연산을 하면 모든 작업을  $O(1)$ 에 처리할 수 있습니다.
  - ✓ 모든 송어를 한 번씩만 처리하므로  $O(N)$ 에 문제를 해결할 수 있습니다.



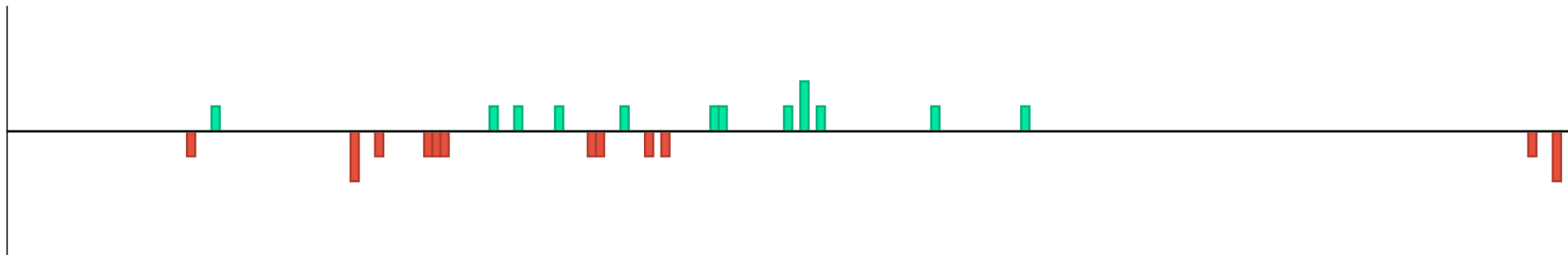


## G. GO! GO! MARU

#simulation, #data\_structures, #priority\_queue

출제진 의도 - **Medium**

- ✓ 출제자: kckyoung2
- ✓ 제출: 27번, 정답: 13명 (정답률 48.1%), 처음 푼 사람: fermion5, 25분





## G. GO! GO! MARU

- ✓ Subtask 1 (20점):  $N, Q \leq 1000$ 
  - ✓ 1, 2번 쿼리에 대해서 인접한 모든 정점을 체크하면 됩니다.
  - ✓ 3번 쿼리에 대해서 단순히 체류 시간을  $t$ 만큼 증가시키면 됩니다.
  - ✓ 매 쿼리마다 최대  $N$ 만큼 탐색하므로  $O(NQ)$ 에 해결할 수 있습니다.



## G. GO! GO! MARU

- ✓ Subtask 2 (80점): 제한 조건 없음 (main solution)
  - ✓ 우선순위 큐와 set을 이용해서 쿼리를 처리해 봅시다.
  - ✓  $N$ 개의 모든 정점에 우선순위 큐, set,  $t$ 를 저장하도록 합시다.
  - ✓ 먼저 2번 쿼리는 단순히 set에서 find()를 사용하여  $O(\log N)$ 에 처리할 수 있습니다.
  - ✓ 3번 쿼리가 들어왔을 때 인접한 모든 정점에 대해서 우선순위 큐를 갱신하는 방법을 생각해봅시다.
  - ✓ 위와 같이 처리할 경우 성계 그래프(Star graph)에서 매번  $N - 1$ 의 우선순위 큐를 업데이트 하게 되므로 시간초과를 받게 됩니다.
  - ✓  $N - 1$ 개의 우선순위 큐를 모두 갱신할 필요가 있을까요?
  - ✓ 체류 시간을 저장하는  $t$  배열을 만들어 두고 1번 쿼리가 들어올 때만 체크해봅시다.
  - ✓ 우선순위 큐에서 top에 있는 값과  $t$  배열에 있는 값을 비교해서 같을 경우 갱신이 되지 않았음을 알 수 있습니다. 만약 같지 않을 경우 갱신된 값을 알 수 있습니다.



## G. GO! GO! MARU

- ✓ 위 과정을 갱신되지 않은 값이 나올 때까지 반복하면 됩니다.
- ✓ Subtask 2 - 시간복잡도 계산
  - ✓ 정점  $A$ 의 업데이트로 인해 연결된 정점  $B$ 의 우선순위 큐에서 상태 갱신이 유의미하게 발생하려면 결국 정점 간의 이동이 필요합니다.
  - ✓ 전체 쿼리 중 이동 횟수의 상한이  $O(Q)$ 로 제한되므로, 전체 과정에서  $t$ 와 일치하지 않는 낡은 값을 꺼내고 다시 삽입하는 총 횟수 역시  $O(Q)$ 에 비례하게 됩니다.
  - ✓ 결과적으로 1번 쿼리 하나를 처리하는 데 소요되는 평균 시간은  $O(\log N)$ 으로 수렴하며,  $O(Q \log N)$ 에 문제를 해결할 수 있습니다.
- ✓ Subtask 2 (80점): 제한 조건 없음 (other solution)
  - ✓ 루트질, 셋질로도 문제를 해결할 수 있습니다.

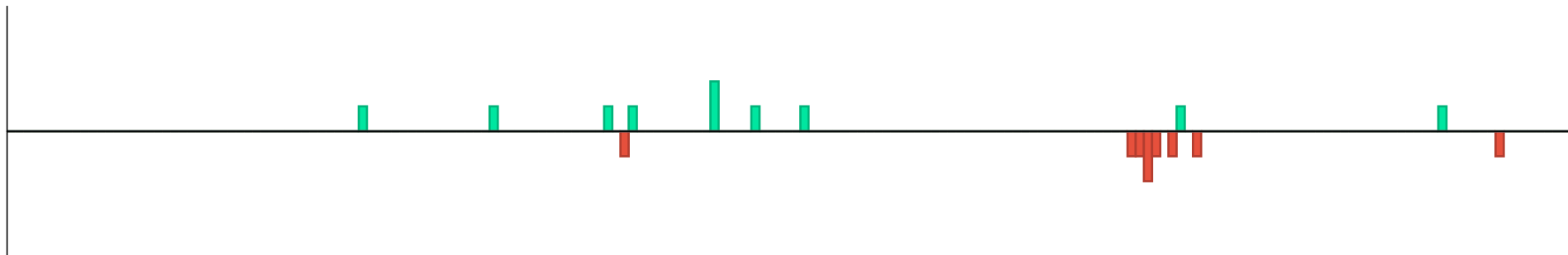


## H. Handle's rule

## #dp, #prefix\_sum, #combinatorics

## 출제진 의도 - Hard

- ✓ 출제자: kckyoung2
- ✓ 제출: 19번, 정답: 10명 (정답률 52.6%), 처음 푼 사람: shihm1212, 43분





## H. Handle's rule

- ✓ Subtask 1 (5점):  $K = 2; R \leq 2$ 
  - ✓ 길이가 1인 핸들의 개수는  $N$ 이 됩니다.
  - ✓ 길이가 2인 핸들을 만들 때 이전과 다른 문자가 와야 합니다.
  - ✓ 각  $N$ 개의 문자에 대해서 다른 문자가 오는 경우의 수는 총  $N - 1$ 가지가 있습니다.
  - ✓ 따라서  $N \times (N - 1)$ 개가 됩니다.
  
- ✓ Subtask 2 (15점):  $K = 2$ 
  - ✓ 길이가  $i \geq 2$ 인 핸들의 개수를 구하는 방법을 일반화하면 다음과 같이 쓸 수 있습니다.
  - ✓ 길이가  $i$ 인 핸들의 개수 = 길이가  $i - 1$ 인 핸들의 개수  $\times (N - 1)$
  - ✓ 따라서 총  $O(R)$ 에 문제를 해결할 수 있습니다.



## H. Handle's rule

- ✓ Subtask 3 (20점):  $N, K, R \leq 200$ 
  - ✓ 다음과 같이 상태공간을 정의합니다.
  - ✓  $dp[i][c][k] :=$  마지막으로 온 문자  $c$ 가  $k$ 번 반복되고 현재 길이가  $i$ 인 핸들의 개수
  - ✓ 다음으로 오는 문자가  $c$ 일 경우는 1가지이며  $k + 1 < K$ 이기만 하면 됩니다.
  - ✓ 다음으로 오는 문자가  $c$ 가 아닐 경우는  $N - 1$ 가지이며 바로 전이하면 됩니다.
  - ✓ 이때 각 상태에 대해서 모든 다른 문자  $N - 1$ 에 대해서 일일이 전이할 경우  $O(N^2KR)$ 이 되어 시간 내에 해결할 수 없습니다.
  - ✓ 전이를 빠르게 하기 위해 미리 전이할 상태의 경우의 수를 모두 더해놓을 수 있습니다.
  - ✓  $i - 1$ 번째 에서  $sum = \sum cnt_c; cnt_c = \sum_k dp[i][c][k]$ 로 두고 미리 계산한 값을 사용합니다.
  - ✓  $dp[i][c][1] = sum - cnt_c$ 로 다른 문자가 오는 경우를  $O(1)$ 에 처리할 수 있습니다.
  - ✓ 따라서  $O(NKR)$ 에 문제를 해결할 수 있습니다.



## H. Handle's rule

- ✓ Subtask 4 (20점):  $K, R \leq 2000$ 
  - ✓ Subtask 3에서 차원을 줄이는 방법을 생각해 봅시다.
  - ✓ 예를 들어 어떤 문자  $a$ 와  $b$ 가 있다고 할 때, 이 둘의 전이는 완전히 똑같습니다.
  - ✓ 즉, 어떤 문자라는 정보는 불필요하며 제거할 수 있습니다.
  - ✓  $dp[i][k] :=$  마지막으로 온 문자가  $k$ 번 반복되고 현재 길이가  $i$ 인 핸들의 개수
  - ✓ Subtask 3에서와 마찬가지로 똑같은 문자를 추가하는 경우는 1가지 입니다.
  - ✓ 다른 문자를 추가하는 경우는 각 문자마다  $N - 1$ 가지가 됩니다.
  - ✓ 따라서 다음과 같은 전이식을 세울 수 있습니다
  - ✓  $dp[i][1] = (N - 1) \sum_{j=1}^{K-1} dp[i-1][j]$
  - ✓ Subtask 3과 비슷하게 미리 값을 더해놓으면 전이를  $O(1)$ 에 처리할 수 있습니다.
  - ✓ 따라서  $O(KR)$ 에 문제를 해결할 수 있습니다.





## H. Handle's rule

- ✓ Subtask 5 (20점):  $N, R \leq 2000$ 
  - ✓ Subtask 3에서 차원을 줄이는 또 다른 방법을 생각해 봅시다.
  - ✓ 문자를 하나씩 추가하는 것이 아니라, 연속된 문자로 구성된 문자열 단위로 추가한다면, 반복 횟수에 대한 정보는 불필요하며 제거할 수 있습니다.
  - ✓  $dp[i][c] :=$  마지막으로 온 문자가  $c$ 이고 현재 길이가  $i$ 인 핸들의 개수
  - ✓  $sum[i][c] :=$  마지막으로 온 문자가  $c$ 가 아니면서 길이가  $[0, i]$ 인 핸들의 개수
  - ✓ 길이가  $i - j$  ( $1 \leq j \leq K$ )이고 마지막 문자가  $c$ 가 아닌 핸들 뒤에  $c$ 를  $j$ 개 통째로 이어 붙여서  $dp[i][c]$ 를 만듭니다.
  - ✓ 전이식을  $dp[i][c] = sum[i - 1][c] - sum[i - K][c]$ 와 같이 둘 수 있습니다.
  - ✓ 누적 합의 계산은  $sum[i][c] = sum[i - 1][c] + \sum_{c \neq c'} dp[i][c']$ 로 둘 수 있습니다.
  - ✓ 따라서  $O(NR)$ 에 문제를 해결할 수 있습니다.



## H. Handle's rule

- ✓ Subtask 6 (40점): 제한 조건 없음 (main solution)
  - ✓ Subtask 4, 5를 통해서 마지막 문자 정보  $c$ 와 반복 횟수  $k$ 가 불필요한 정보임을 알 수 있었습니다.
  - ✓  $dp[i] :=$  길이가  $i$ 인 핸들의 개수
  - ✓  $sum[i] := \sum_{j=1}^i dp[j]$
  - ✓ 위와 같이 상태공간과 누적합을 정의합니다.
  - ✓ 전이식을  $dp[i] = (N - 1) \sum_{j=1}^{K-1} dp[i - j]$  로 둘 수 있습니다.
  - ✓ 이는 Subtask 5에서처럼 마지막으로 온 문자가 아닌 어떤 문자  $c$ 가  $j$ 개 연속해서 붙어있는 문자열  $N - 1$ 가지를 추가하는 것과 같습니다.
  - ✓ 여기서,  $\sum_{j=1}^{K-1} dp[i - j]$  는 미리 만들어둔 누적합 배열  $sum[i]$ 를 통해서  $O(1)$ 에 계산할 수 있습니다.
  - ✓ 따라서  $O(R)$ 에 문제를 해결할 수 있습니다.

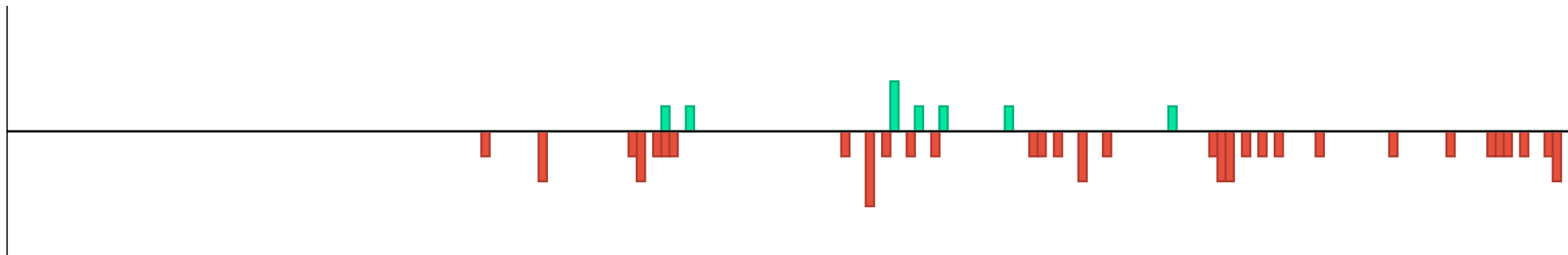


# I. I'm fox, not cat

#geometry, #combinatorics

출제진 의도 - Hard

- ✓ 출제자: kckyoung2
- ✓ 제출: 48번, 정답: 8명 (정답률 16.7%), 처음 푼 사람: akrobatik, 80분





## I. I'm fox, not cat

- ✓ Subtask 1 (20점):  $N \leq 300$ 
  - ✓ 단순히 삼중 반복문을 통해서 모든 세 쌍의 점을 순회합니다.
  - ✓ 일직선 상에 있지 않으면서 모두 동일한 색이 아닌 경우를 세면 됩니다.
  - ✓ 위와 같은 완전탐색을 통해  $O(N^3)$ 에 해결할 수 있습니다.
- ✓ Subtask 2 (80점): 제한 조건 없음 (main solution)
- ✓ Solution 1:  $O(N^2 \log N)$ 
  - ✓ 양의 넓이를 가지는 삼각형이 아니라, 단순히 같은 색상으로만 이루어지지 않은 삼각형의 구한다고 생각해 봅시다.
  - ✓ 하얀 점의 개수를  $W$ , 검은 점의 개수를  $B$ 라고 할 때, 답은  $S = \binom{W}{2} \times B + \binom{B}{2} \times W$ 이 됩니다.



## I. I'm fox, not cat

- ✓ 이 개수에서 일직선 상에 있는 모두 같은 색상이 아닌 세 점의 개수를 빼주면 됩니다.
- ✓ 어떤 점  $i$ 와  $N - 1$ 개의 점들과 각각 이어서 만든 직선을 벡터  $(p, q)$ 로 나타낼 수 있습니다.
- ✓ 위에서 만든 어떤 직선  $l_j$ 에서 하얀 점의 개수를  $W_{i,j}$ 개, 검은 점의 개수를  $B_{i,j}$ 개라고 합니다.
- ✓ 일직선 상에서 모두 같은 색상이 아닌 세 점을 고르는 방법은  $S_{i,j} = \binom{W_{i,j}}{2} \times B_{i,j} + \binom{B_{i,j}}{2} \times W_{i,j}$ 가 됩니다.
- ✓  $T = \sum_{i,j;i \neq j} S_{i,j}$ 로 두었을 때  $T$ 는 모두 같은 색상이 아닌 세 점의 개수의 3배가 됩니다.
- ✓ 똑같은 구성의 삼각형을 중복해서 카운트하기 때문에 최종적으로  $S - \frac{T}{3}$ 이 정답이 됩니다.
- ✓ 같은 직선을 모으는데 set 등의 자료구조를 사용할 수 있고, 총  $N^2$ 번 반복하게 되어  $O(N^2 \log N)$ 에 문제를 해결할 수 있습니다.



## I. I'm fox, not cat

- ✓ Solution 2:  $O(N^2)$ 
  - ✓ 직선을 모으는 것을 조금 더 빨리 처리할 수 있습니다.
  - ✓ 좌표의 범위가  $[-1000, 1000]$ 이므로 단순히 배열을 만들어서 처리할 수 있습니다.
  - ✓ 직선 벡터의  $dx$ 를 0이상의 정수로 고정하면,  $dx$ 의 범위를  $[0, 2000]$ ,  $dy$ 의 범위를  $[-2000, 2000]$ 으로 볼 수 있습니다.
  - ✓ 배열 인덱스로 두기 위해 2000만큼 더한 값인  $[0, 4000]$ 을 다루면 됩니다.
  - ✓ 그 외에는 Solution 1과 동일합니다.
  - ✓ 최종적으로  $O(N^2)$ 에 문제를 해결할 수 있습니다.



## I. I'm fox, not cat

- ✓ Subtask 2 (80점): 제한 조건 없음 (other solution)
  - ✓ 하얀 점 두 개를 잇는 모든 직선을 map 등에 저장합니다.
  - ✓ 검은 점과 하얀 점을 잇는 모든 직선을 map 등에 저장합니다.
  - ✓ 이걸 잘 역산하면 모든 하얀 점이 두 개 이상 포함된 직선에 대해 포함하는 점의 개수를 구할 수 있습니다.
  - ✓ 그 외에는 main solution 처럼 조합으로 잘 계산하면 됩니다.

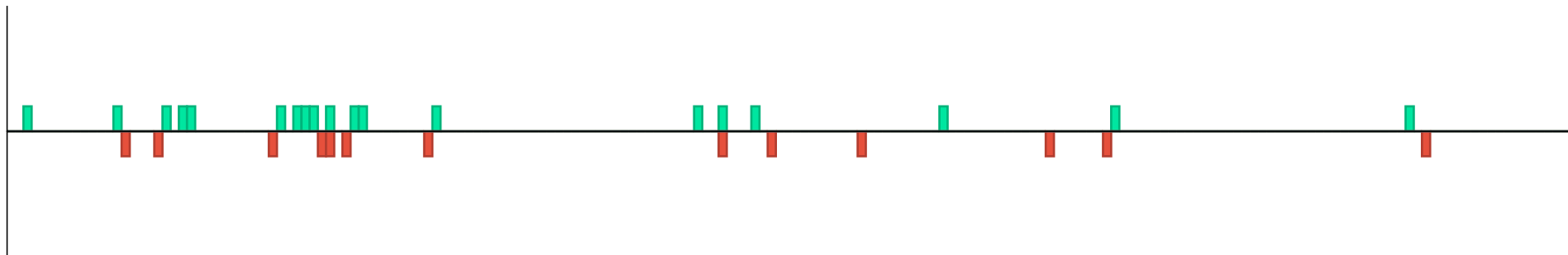


# J. Jaguar

#ad\_hoc, #math

출제진 의도 - **Easy**

- ✓ 출제자: dustin36
- ✓ 제출: 32번, 정답: 19명 (정답률 59.4%), 처음 푼 사람: wlaud, 2분







- ✓ 하루 동안 선택된 재규어의 체온은 1 감소하고, 선택되지 않은 재규어들의 체온은 1 증가합니다.
- ✓ 두 재규어의 체온 차이를 생각해 봅시다. 하루가 지나도 이 차이는 0 또는 2만큼만 변합니다. 따라서 두 체온 차이의 홀짝성은 절대 변하지 않습니다.
- ✓ 모든 체온이 같아지려면 모든 쌍의 체온 차이가 짝수여야 하므로, 처음부터 모든 체온의 홀짝성이 같아야 합니다. 짝수와 홀수가 섞여 있다면 답은  $-1$ 입니다.
- ✓ 이제 모든 체온의 홀짝성이 같다고 합시다.
- ✓ 최솟값을  $m$ 이라고 두고, 각 재규어에 대해  $d_i = \frac{a_i - m}{2}$ 로 정의합니다.
- ✓ 총  $K = d_1 + d_2 + \dots + d_N$ 일 동안  $i$ 번째 재규어를 정확히  $d_i$ 번 선택한다고 합시다.
- ✓ 그러면 최종 체온은  $a_i + K - 2d_i = m + K$ 가 되어 모든 재규어의 체온이 같아집니다.
- ✓ 또한 최솟값을 가진 재규어보다 체온이  $a_i - m$ 만큼 높은 재규어는 차이를 줄이기 위해 적어도  $\frac{a_i - m}{2}$ 번 선택되어야 합니다.



## J. Jaguar

- ✓ 따라서 위 방법보다 더 적은 날짜로는 불가능합니다.
- ✓ 최종적으로 정답은 모든 체온의 홀짝성이 같을 때  $\frac{a_i - m}{2}$ 의 합이고, 그렇지 않으면  $-1$ 입니다.
- ✓ 선형적으로 탐색하므로  $O(N)$ 에 문제를 해결할 수 있습니다.

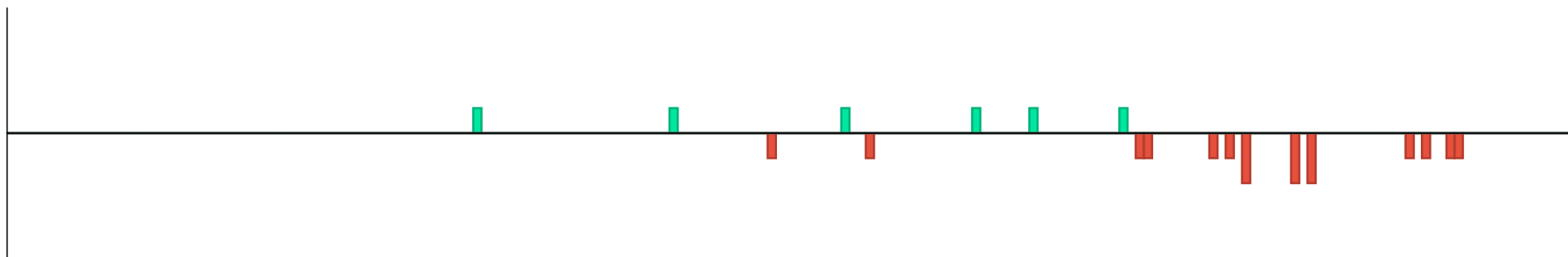


## K. kENN3N's Trekking

#graph\_traversal

출제진 의도 - Hard

- ✓ 출제자: kckyoung2
- ✓ 제출: 22번, 정답: 6명 (정답률 27.3%), 처음 푼 사람: akrobatik, 57분





## K. kENN3N's Trekking

- ✓ Subtask 1 (20점):  $N \leq 100$ 
  - ✓ 먼저 kENN3N의 초기 위치가 트랩의 감시 범위에 속할 경우 0을 출력해야 합니다.
  - ✓ 그 외에 경우에는 BFS, DFS 등의 그래프 탐색을 진행합니다.
  - ✓ 다른 트랩에게 감시되지 않는 트랩이 인접할 경우 제거할 수 있습니다.
  - ✓ 트랩을 제거할 때마다 매번 처음부터 그래프 탐색을 다시 진행하면 됩니다.
  - ✓ 이론상 제거할 수 있는 트랩의 최대 개수가  $2n - 1$ 개 입니다.
  - ✓ 따라서  $O(N^3)$ 에 문제를 해결할 수 있습니다.



## K. kENN3N's Trekking

- ✓ Subtask 2 (80점): 제한 조건 없음 (main solution)
  - ✓ Subtask 1에서 어떻게 하면 재탐색을 효율적으로 할 수 있을지 생각해 봅시다.
  - ✓ 트랩의 감시 범위에 속해서 갈 수 없었던 칸을 대각선 별로 따로 저장해 둡니다.
  - ✓ 트랩을 제거함으로써 어떤 대각선의 감시가 사라졌을 때, 그 대각선에서 가지 못했던 칸을 전부 탐색하도록 하면 됩니다.
  - ✓ 한 번이라도 도달한 칸을 다시 탐색하는 경우는 없습니다.
  - ✓ 따라서  $O(N^2)$ 에 문제를 해결할 수 있습니다.